

Visualização de dados e EDA

Eduardo Adame

Ciência de Dados

10 de abril de 2026



Por que visualizar?



Antes de modelar, precisamos **entender** os dados — e o olho humano é extraordinariamente bom em detectar padrões, outliers e estruturas que estatísticas resumidas escondem.

Quarteto de Anscombe (1973)

- Quatro datasets com **mesma média, variância e correlação**
- Estruturas completamente diferentes
- Invisíveis sem visualização

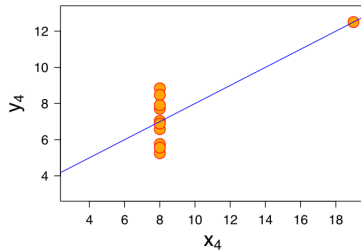
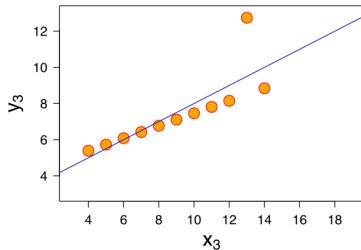
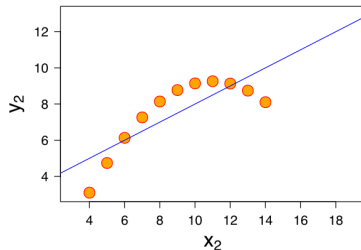
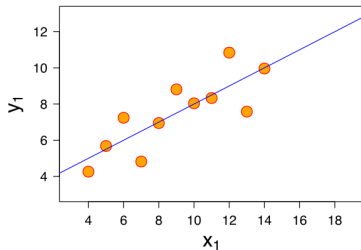
Datasaurus Dozen (2017)

- 13 datasets com **estatísticas idênticas**
- Formas radicalmente distintas
- Moral: `df.describe()` não é suficiente

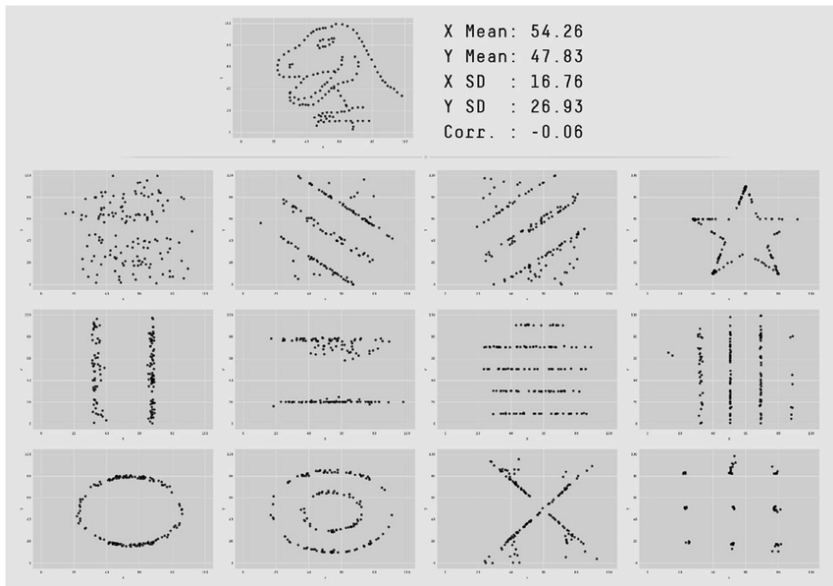
Regra de ouro da EDA

Nunca ajuste um modelo antes de plotar os dados.

Por que visualizar? Quarteto de Anscombe (1973)



Por que visualizar? Datasaurus Dozen (2017)



O ecossistema de visualização em Python



Baseadas em Matplotlib

- matplotlib — controle total, verboso
- pandas .plot — atalho rápido para DataFrames
- seaborn — estatística + estética

Independentes

- plotly — interativo, web-ready
- altair — gramática declarativa (Vega-Lite)

Qual usar?

Exploração rápida: pandas ou seaborn. Publicação: matplotlib. Dashboards: plotly. Artigos e gramática limpa: altair.



Listing 1: Tipos básicos com .plot

```
1 import pandas as pd
2 df["nota"].plot(kind="hist", bins=20, title="Distribuição de notas")
3 df["nota"].plot(kind="box")
4 df.plot(kind="scatter", x="idade", y="nota")
5 df["turma"].value_counts().plot(kind="bar")
```

Listing 2: Atalhos convenientes

```
1 df["nota"].plot.hist(bins=20)
2 df["nota"].plot.kde()           # densidade estimada
3 df.plot.scatter(x="x", y="y")
```

- Retorna um objeto matplotlib Axes — customizável depois
- kind: 'line', 'bar', 'barh', 'hist', 'box', 'kde', 'scatter', 'pie'



Listing 3: Comparar distribuições por grupo

```
1 df.groupby("turma")["nota"].plot(kind="kde", legend=True)
2 # Múltiplas colunas lado a lado
3 df[["nota_p1", "nota_p2", "nota_p3"]].plot(kind="box")
4 # Série temporal
5 df.set_index("data")["vendas"].plot(kind="line")
```

- `subplots=True` cria um painel por coluna
- `figsize=(largura, altura)` controla o tamanho
- Passe `ax=ax` para compor com Matplotlib

Quando usar `.plot`

Ideal para exploração rápida e prototipagem. Para publicação, migre para Matplotlib ou Seaborn.



Listing 4: Interface orientada a objetos (recomendada)

```
1 import matplotlib.pyplot as plt
2 fig, ax = plt.subplots(figsize=(8, 4))
3 ax.scatter(df["idade"], df["nota"], alpha=0.5, color="#5B2D8E")
4 ax.set_xlabel("Idade")
5 ax.set_ylabel("Nota final")
6 ax.set_title("Nota vs. Idade")
7 ax.axhline(6.0, color="red", linestyle="--", label="Aprovação")
8 ax.legend()
9 fig.tight_layout()
```

- `fig` controla tamanho, resolução e exportação
- `ax` controla o conteúdo do gráfico
- Prefira a interface OO à interface `plt.xxx()` em scripts



Listing 5: Subplots com layout automático

```
1 fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(10, 7))
2 axes[0, 0].hist(df["nota_p1"].dropna(), bins=20, color="#5B2D8E")
3 axes[0, 1].hist(df["nota_p2"].dropna(), bins=20, color="#E8650A")
4 axes[1, 0].hist(df["nota_p3"].dropna(), bins=20, color="#2D8E5B")
5 axes[1, 1].boxplot([df["nota_p1"].dropna(),
6                     df["nota_p2"].dropna(),
7                     df["nota_p3"].dropna()])
8 fig.suptitle("Distribuição por prova", fontsize=14)
9 fig.tight_layout()
```

Listing 6: Layout com proporções customizadas

```
1 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4),
2                               gridspec_kw={"width_ratios": [2, 1]})
```



Listing 7: Salvando em diferentes formatos

```
1 fig.savefig("figura.png", dpi=150, bbox_inches="tight")
2 fig.savefig("figura.pdf", bbox_inches="tight")      # vetorial
3 fig.savefig("figura.svg", bbox_inches="tight")      # web
```

- dpi=150 ou dpi=300 para qualidade de publicação
- bbox_inches="tight" evita cortes nas bordas
- transparent=True para fundo transparente
- PDF e SVG são **vetoriais** — preferíveis para artigos

Dica de reprodutibilidade

Salve as figuras programaticamente no pipeline — nunca use “Print Screen”.



Seaborn é construído sobre Matplotlib e oferece **tipos de gráfico estatísticos** com uma API de alto nível.

Listing 8: Importando e configurando tema

```
1 import seaborn as sns
2 sns.set_theme(style="whitegrid", palette="muted")
```

Distribuições

- `histplot` — histograma + KDE
- `kdeplot` — densidade
- `boxplot` / `violinplot`
- `ecdfplot` — função acumulada

Relações e categorias

- `scatterplot` / `lineplot`
- `barplot` — média + IC
- `heatmap` — correlação
- `pairplot` — matriz de dispersão



Listing 9: Histograma com KDE e separação por grupo

```
1 import seaborn as sns
2 sns.histplot(data=df, x="nota", hue="turma",
3              kde=True, bins=20, alpha=0.5)
4 sns.boxplot(data=df, x="turma", y="nota",
5             order=["A", "B", "C"])
6 sns.violinplot(data=df, x="turma", y="nota",
7               inner="quartile", palette="muted")
```

- hue separa grupos por cor automaticamente
- violinplot mostra distribuição e quartis simultaneamente
- Todos retornam Axes — customizáveis com Matplotlib



Listing 10: Scatter com regressão e correlação

```
1 sns.regplot(data=df, x="idade", y="nota", scatter_kws={"alpha": 0.4})
2 # Mapa de calor da correlação
3 corr = df[["nota_p1", "nota_p2", "nota_p3", "idade"]].corr()
4 sns.heatmap(corr, annot=True, fmt=".2f",
5             cmap="coolwarm", vmin=-1, vmax=1)
6 # Matriz de dispersão completa
7 sns.pairplot(df[["nota_p1", "nota_p2", "nota_p3"]], diag_kind="kde")
```

Interpretando o heatmap

Valores próximos de $+1$: correlação positiva forte. Próximos de 0 : sem relação linear. Próximos de -1 : correlação negativa forte.



Listing 11: Replicar gráfico por subgrupo

```
1 g = sns.FacetGrid(df, col="turma", height=3, aspect=1.2)
2 g.map(sns.histplot, "nota", bins=15, kde=True)
3 g.set_axis_labels("Nota", "Frequência")
4 g.set_titles("Turma {col_name}")
```

Listing 12: catplot — interface de alto nível

```
1 sns.catplot(data=df, x="turma", y="nota",
2             hue="aprovado", kind="box",
3             height=4, aspect=1.5)
```

- FacetGrid é a base de catplot, relplot, displot
- Permite criar **painéis condicionais** com uma linha

Plotly — visualizações interativas



Plotly gera gráficos **interativos** baseados em HTML/JavaScript — ideais para dashboards e relatórios web.

Listing 13: Plotly Express — API rápida

```
1 import plotly.express as px
2 fig = px.histogram(df, x="nota", color="turma",
3                   nbins=20, barmode="overlay",
4                   title="Distribuição de notas por turma")
5 fig.show()
6 fig = px.scatter(df, x="idade", y="nota",
7                 color="turma", hover_data=["nome"],
8                 trendline="ols")
9 fig.show()
```

- `hover_data` — dados exibidos ao passar o mouse
- `color`, `facet_col`, `animation_frame` — encoding declarativo



Listing 14: Exportar para HTML e imagem

```
1 fig.write_html("grafico.html")           # interativo
2 fig.write_image("grafico.png", scale=2)  # requer kaleido
3
4 # Customização com update_layout
5 fig.update_layout(
6     template="plotly_white",
7     font=dict(family="Arial", size=13),
8     legend=dict(orientation="h", y=-0.2),
9 )
10 fig.update_traces(marker_opacity=0.6)
```

Quando usar Plotly

Relatórios interativos, dashboards (Dash/Streamlit) e quando o leitor precisará explorar os dados. Para publicação estática, prefira Matplotlib ou Seaborn.



Altair implementa a **gramática Vega-Lite**: você declara o *mapeamento* entre dados e elementos visuais, e a biblioteca cuida do resto.

Listing 15: Conceito: encoding de canais

```
1 import altair as alt
2 alt.Chart(df).mark_point().encode(
3     x="idade:Q",      # Q = quantitativo
4     y="nota:Q",
5     color="turma:N",  # N = nominal
6     tooltip=["nome", "nota", "turma"]
7 )
```



Tipos de dados

- Q — quantitativo
- N — nominal (categórico)
- O — ordinal
- T — temporal

Marcas (mark_*)

- point, line, bar
- area, rect, rule
- boxplot, errorband



Listing 16: Camadas e painéis

```
1 base = alt.Chart(df)
2 pontos = base.mark_point(opacity=0.5).encode(
3     x="idade:Q", y="nota:Q", color="turma:N"
4 )
5 linha = base.mark_rule(color="red").encode(
6     y=alt.datum(6.0)           # linha de aprovação
7 )
8 grafico = (pontos + linha).properties(width=400, height=300)
9 # Facetamento
10 grafico.facet(column="turma:N")
```



Listing 17: Interatividade com uma linha

```
1 selecao = alt.selection_point(fields=["turma"])
2 pontos.add_params(selecao).encode(
3     opacity=alt.condition(selecao, alt.value(1), alt.value(0.1))
4 )
```

Colormaps — escolha certa para cada dado



Sequencial

Dados ordenados com zero natural

- viridis, plasma
- Blues, YlOrRd
- Claro → escuro

Divergente

Dados com ponto médio significativo

- RdBu, coolwarm
- PuOr, BrBG
- Ex: correlações, desvios

Qualitativo

Categorias sem ordem

- tab10, Set2
- Seaborn ‘muted’
- Ex: turmas, classes

Acessibilidade

Evite jet e rainbow — enganam o olho e são inacessíveis para daltônicos.
viridis é perceptualmente uniforme e seguro.

Qual gráfico para qual tipo de dado?



Uma variável

- Contínua: histplot, kdeplot, boxplot
- Discreta / categórica: countplot, barplot

Duas variáveis

- Contínua $\times$ contínua: scatter, hexbin, kde2d
- Categórica $\times$ contínua: box, violin, strip
- Categórica $\times$ categórica: heatmap, mosaico

Muitas variáveis

- Matriz: pairplot, heatmap de correlação
- Terceira variável: cor, tamanho, forma, faceta

Temporal

- lineplot, área empilhada
- Sazonalidade: heatmap mês $\times$ ano

EDA — checklist prático



1. Inspecionar

- `df.info()`, `df.describe()`
- Tipos corretos? NaN onde?

2. Univariada

- Distribuição de cada variável
- Outliers visíveis?
- Simetria, caudas, modas

3. Bivariada

- Correlações entre numéricas
- Variáveis numéricas por categoria

4. Multivariada

- `pairplot` ou heatmap
- Adicionar cor/faceta por grupo

5. Temporal (se houver)

- Tendência, sazonalidade
- Quebras estruturais

6. Comunicar

- Título informativo
- Eixos rotulados com unidades
- Colormap adequado

Objetivo da EDA

Gerar hipóteses. A confirmação vem depois, com modelos.

Obrigado!

Alguma dúvida?

Agora vamos para os exercícios!